

SQL Reference Guide - Complete

Eleazar Soto - Data Analytics Portfolio

Level 1 - Basics (Completed)

1. SELECT + FROM

```
SELECT columna1, columna2
FROM tabla;
```

2. WHERE - Filter rows

```
SELECT columna1, columna2
FROM tabla
WHERE condicion;
```

3. IS NULL / IS NOT NULL

```
SELECT columna1 FROM tabla WHERE columna IS NULL;
SELECT columna1 FROM tabla WHERE columna IS NOT NULL;
```

Level 2 - Aggregation (Completed)

4. ORDER BY

```
SELECT columna1, columna2
FROM tabla
ORDER BY columna ASC; -- or DESC
```

5. COUNT / SUM / AVG / MAX / MIN

```
SELECT COUNT(*) AS total
FROM tabla
WHERE condicion;
```

6. GROUP BY + HAVING

```
SELECT columna, COUNT(*)
FROM tabla
GROUP BY columna
HAVING COUNT(*) > 10
ORDER BY COUNT(*) DESC;
```

7. LIKE - Pattern matching

```
WHERE columna LIKE 'valor%'; -- starts with
WHERE columna LIKE '%valor%'; -- contains
WHERE columna LIKE '%valor'; -- ends with
```

8. IN - Multiple values

```
SELECT columna1  
FROM tabla  
WHERE columna IN ('valor1', 'valor2', 'valor3');
```

9. BETWEEN - Range

```
SELECT columna1  
FROM tabla  
WHERE columna BETWEEN valor1 AND valor2;
```

Level 3 - Joins (Completed)

10. INNER JOIN - 2 tables

```
SELECT t1.columna, t2.columna
FROM tabla1 t1
JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun;
```

11. INNER JOIN - 3 tables

```
SELECT t1.columna, t2.columna, t3.columna
FROM tabla1 t1
JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun
JOIN tabla3 t3 ON t2.columna_comun = t3.columna_comun;
```

12. LEFT JOIN

```
SELECT t1.columna, t2.columna
FROM tabla1 t1
LEFT JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun;
```

13. LEFT JOIN - Find unmatched records

```
SELECT t1.columna
FROM tabla1 t1
LEFT JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun
WHERE t2.columna IS NULL;
```

Level 4 - CASE (Completed)

14. CASE - Conditionals

```
SELECT columna,
CASE
    WHEN condicion1 THEN 'resultado1'
    WHEN condicion2 THEN 'resultado2'
    ELSE 'resultado_default'
END AS nombre_columna
FROM tabla;
```

15. CASE + JOIN + GROUP BY

```
SELECT t1.columna,
CASE
    WHEN COUNT(*) > 10 THEN 'VIP'
    WHEN COUNT(*) BETWEEN 5 AND 10 THEN 'Regular'
    ELSE 'New'
END AS categoria
FROM tabla1 t1
JOIN tabla2 t2 ON t1.columna_comun = t2.columna_comun
GROUP BY t1.columna;
```

16. CASE with IN

```
SELECT columna,  
CASE  
    WHEN columna IN ('valor1', 'valor2') THEN 'Grupo A'  
    ELSE 'Grupo B'  
END AS categoria  
FROM tabla;
```

Level 5 - Subqueries (Completed)

17. Scalar subquery - single value

```
SELECT columna
FROM tabla
WHERE columna > (SELECT AVG(columna) FROM tabla);
```

18. Subquery with IN - multiple values

```
SELECT columna
FROM tabla1
WHERE columna IN (
    SELECT columna FROM tabla2 WHERE condicion
);
```

19. Nested subquery - subquery inside subquery

```
SELECT t1.columna, COUNT(*) AS total
FROM tabla1 t1
JOIN tabla2 t2 ON t1.id = t2.id
GROUP BY t1.columna
HAVING COUNT(*) = (
    SELECT MAX(cnt) FROM (
        SELECT COUNT(*) AS cnt FROM tabla2 GROUP BY id
    )
);
```

Level 6 - UNION (Completed)

20. UNION - Combine results, remove duplicates

```
SELECT columna FROM tabla1
UNION
SELECT columna FROM tabla2;
```

21. UNION with labels

```
SELECT nombre, 'TypeA' AS Tipo FROM tabla1
UNION
SELECT nombre, 'TypeB' AS Tipo FROM tabla2;
```

22. UNION ALL - Keeps duplicates

```
SELECT columna FROM tabla1
UNION ALL
SELECT columna FROM tabla2;
```

Clause Order - Always follow this

SELECT -> FROM -> JOIN -> WHERE -> GROUP BY -> HAVING -> ORDER BY -> LIMIT

Key Rules

Rule	Example
Text with single quotes	WHERE City = 'London'
Numbers without quotes	WHERE UnitPrice > 20
Filter groups	HAVING not WHERE
NULL comparison	IS NULL not = NULL
Unique alias per table	c, o, p, ca, s, sh, e
CASE uses real columns	WHEN UnitPrice < 10 not alias
CASE order	WHEN condition THEN result
UNION columns must match	Same number and type of columns
Subquery runs first	Inner query executes before outer

Northwind - Table Connections

Customers ---- Orders ---- Shippers
CustomerID ShipVia = ShipperID

Products ---- Categories Products ---- Suppliers
CategoryID SupplierID

Employees ---- Orders
EmployeeID

Progress Summary

Level	Topic	Status
1	SELECT, FROM, WHERE, IS NULL	Completed
2	ORDER BY, COUNT, GROUP BY, HAVING	Completed
3	INNER JOIN, LEFT JOIN	Completed
4	CASE statements	Completed
5	Subqueries (scalar, IN, nested)	Completed
6	UNION	Completed
7	Window Functions, CTEs	Next